8-Puzzle Problem - Python Program

A* search algorithm using Manhattan distance as the heuristic. The blank tile is represented by 0.

```
import heapq

# Goal state: 0 represents the blank space
GOAL = (1, 2, 3,
        4, 5, 6,
        7, 8, 0)


def manhattan(state):
    distance = 0

    for i, tile in enumerate(state):
        if tile != 0:
            goal_index = GOAL.index(tile)
            row1, col1 = divmod(i, 3)
            row2, col2 = divmod(goal_index, 3)
            distance += abs(row1 - row2) + abs(col1 - col2)

    return distance


def get_neighbors(state):
    neighbors = []
    zero = state.index(0)
    row, col = divmod(zero, 3)
    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    for dr, dc in moves:
        new_row = row + dr
        new_col = col + dc

        if 0 <= new_row < 3 and 0 <= new_col < 3:
            new_index = new_row * 3 + new_col
            new_state = list(state)
            new_state[zero], new_state[new_index] = (
                new_state[new_index], new_state[zero]
            )
            neighbors.append(tuple(new_state))

    return neighbors


def solve_puzzle(start):
    priority_queue = []
    heapq.heappush(priority_queue, (manhattan(start), 0, start))
    parent = {start: None}
    cost = {start: 0}

    while priority_queue:
        _, g, current = heapq.heappop(priority_queue)

        if g != cost[current]:
            continue

        if current == GOAL:
            path = []
            while current is not None:
                path.append(current)
                current = parent[current]
            return path[::-1]

        for neighbor in get_neighbors(current):
            new_cost = g + 1

            if neighbor not in cost or new_cost < cost[neighbor]:
                cost[neighbor] = new_cost
                parent[neighbor] = current
                f = new_cost + manhattan(neighbor)
                heapq.heappush(
                    priority_queue, (f, new_cost, neighbor)
                )

    return None
def print_state(state):
    for i in range(0, 9, 3):
```

```
        print(*state[i:i + 3])
    print()


# Example initial state
start = (1, 2, 3,
         4, 0, 6,
         7, 5, 8)

solution = solve_puzzle(start)

if solution:
    print("Solution found!")
    print("Number of moves:", len(solution) - 1)
    for step, state in enumerate(solution):
        print("Step", step)
        print_state(state)
else:
    print("No solution found.")
```

How it works: A* selects the state with the lowest *f(n) = g(n) + h(n)*, where *g(n)* is the number of moves so far and *h(n)* is the Manhattan distance to the goal state.